

LED Lab 2

Welcome to the second lab in our series on Light-Emitting Diodes (LEDs)! Building upon our previous work, this lab will explore how to use a single LED for meaningful communication.

The primary objective of this lab is for students to:

- **Understand the concept of digital encoding using a simple on/off system.**
- **Learn about Morse Code and its application in emergency signaling.**
- **Practice fundamental coding concepts (variables, functions, loops, and delays) to control an electronic component.**

This project focuses on creating the internationally recognized S.O.S. distress signal using blinking light, which is a perfect demonstration of how simple electronic states (on/off) can transmit complex information.

Morse Code is a character encoding system that uses sequences of short and long signals called "dits" (short) and "dahs" (long) to represent letters and numbers. This binary, on/off nature makes it uniquely suited for a digital system like our microcontroller, allowing us to transmit characters simply by turning an LED on and off in the correct pattern.

Lab 2 – Singular LED S.O.S. Morse Code

The S.O.S. distress signal in International Morse Code is: . . . - - - . . .

Dit (. . .): A short flash of light.

Dah (- - -): A long flash of light which is typically three times the duration of a dit.

Enter the Code

Open your Arduino IDE and type in the code below.

```
// Define the digital pin the LED is connected to.
const int ledPin = 10;

// Define the base time unit for a 'dit' (short signal) in milliseconds.
// All other timing is based on this value for consistency.
const int ditDuration = 100;

// --- Helper Functions for Morse Code Timing ---
// Function to perform a 'dit' (short flash)
```

```

void flashDit() {
    digitalWrite(ledPin, HIGH); // Turn the LED ON
    delay(ditDuration);        // Keep it on for the dit duration
    digitalWrite(ledPin, LOW); // Turn the LED OFF
    delay(ditDuration);        // Short gap between signals within the same letter (1 unit of time)
}

```

// Function to perform a 'dah' (long flash)

```

void flashDah() {
    digitalWrite(ledPin, HIGH); // Turn the LED ON
    delay(ditDuration * 3);     // Keep it on for three times the dit duration
    digitalWrite(ledPin, LOW); // Turn the LED OFF
    delay(ditDuration);        // Short gap between signals within the same letter (1 unit of time)
}

```

// Function to pause between letters

```

void interLetterGap() {
    // A longer gap (3 units of time) to separate the letters S and O, and O and S.
    // Since flashDit/flashDah already include 1 unit of time, we only need 2 more units.
    delay(ditDuration * 2);
}

```

// --- Main Setup and Loop ---

```

void setup() {
    // Initialize the designated pin as an output pin.
    pinMode(ledPin, OUTPUT);
}

```

```

void loop() {
    // The 'S' in S.O.S. is three dits: ...
    flashDit();
    flashDit();
    flashDit();

    interLetterGap(); // Pause between 'S' and 'O'
}

```

```

// The 'O' in S.O.S. is three dahs: ---
flashDah();

flashDah();

flashDah();

interLetterGap(); // Pause between 'O' and 'S'

// The second 'S' in S.O.S. is three dits: ...
flashDit();

flashDit();

flashDit();

// Wait for a long period (7 units of time, or more) before repeating the S.O.S. signal.
// The original 5000ms (5 seconds) is a clear and effective choice.
delay(5000);
}

```

Once all the code has been copied, press the Verify/Compile button at the top left of the IDE to make sure there are no errors in your code. If this is successful, click the upload button to upload the code to your Arduino. If you have done everything right, you should now see the colored LED on the breadboard flashing on and off every second. Let's look at the code and the hardware to find out how they both work.

Mini Lab: Singular LED S.O.S. Morse Code ([Code Overview](#))

This approach uses a fundamental programming concept called **function abstraction** to organize the different parts of the S.O.S. pattern.

What is function abstraction?

Function abstraction is a core principle in computer programming that refers to the process of hiding the complex implementation details of a specific task behind a simple, easy-to-use interface such as the function's name. Essentially, it allows you to define a task once and then call that task by name whenever you need it, without worrying about *how* the task is accomplished every time.

In the lab code, the functions “**flashDit()**”, “**flashDah()**”, and “**interLetterGap()**” are all examples of function abstractions.

Definitions and Timing Control

The first section establishes key constants and variables used throughout the program:

```
const int ledPin = 10;
```

This line creates a **constant integer** variable named `ledPin` and assigns it the value 10. Using the `const` keyword is best practice because it tells the computer that the LED's pin connection **will not change** during the program's execution.

```
const int ditDuration = 100;
```

This is the most important variable for controlling the signal speed. It defines the base **time unit** (a "dit" or short signal) in milliseconds. By defining this, we can easily change the speed of the entire S.O.S. signal by modifying only this one line of code!

Helper Functions

Instead of repeating blocks of code, we create three **helper functions** that handle the specific timing for each element of the Morse Code signal. With the first one being: **Flashing a Dit (. .)**

```
void flashDit() {  
    digitalWrite(ledPin, HIGH); // LED ON  
    delay(ditDuration);      // Wait for 1 unit of time (100ms)  
    digitalWrite(ledPin, LOW); // LED OFF  
    delay(ditDuration);      // Wait for 1 unit of time (Intra-character gap)  
}
```

A **function** is a reusable block of code that performs a specific task.

The **flashDit()** function executes the commands necessary to create a short flash.

The LED is turned HIGH (ON) for **ditDuration** and then turned LOW (OFF) for the same duration. This gap is the standard **intra-character space** used between the dots and dashes that make up a letter.

The third helper function that we use is :**Flashing a Dah** (– – –)

```
void flashDah() {  
    digitalWrite(ledPin, HIGH); // LED ON  
    delay(ditDuration * 3); // Wait for 3 units of time (300ms)  
    digitalWrite(ledPin, LOW); // LED OFF  
    delay(ditDuration); // Wait for 1 unit of time (Intra-character gap)  
}
```

The flashDah() function creates the long flash. We use simple multiplication which is (* 3) to make the dah three times longer than the dit, which also adheres to the standard Morse Code ratio.

The last helper function we use is: **Inter-Letter Gap**

```
void interLetterGap() {  
    // We need 3 units of time between letters. flashDit/flashDah already include 1 unit of time.  
    delay(ditDuration * 2);  
}
```

In Morse Code, the space between letters (like the space between the **S** and the **O**) should be **three** units of time. Since the previous flashDit or flashDah call already created one unit of space, we only need to add two more units.

The **setup()** function remains standard, initializing the designated pin as an output pin.

```
void setup() {  
    pinMode(ledPin, OUTPUT);  
}
```

The core logic is contained in the **loop()** function.

```
void loop() {  
    // S = dot dot dot  
    flashDit();  
    flashDit();  
    flashDit();  
    interLetterGap(); // Pause between 'S' and 'O'  
  
    // O = dash dash dash  
    flashDah();  
    flashDah();  
    flashDah();  
    interLetterGap(); // Pause between 'O' and 'S'  
  
    // S = dot dot dot  
    flashDit();  
    flashDit();  
    flashDit();  
  
    // Wait 5 seconds before repeating the S.O.S. signal.  
    delay(5000);  
}
```

Since we used helper functions, the loop() function is incredibly simple to read. It clearly and directly maps to the S.O.S. pattern which is (. . . - - - . . .).

The S.O.S. Pattern:

- The first three flashDit() calls signal the letter **S**.
- interLetterGap() creates the required pause.
- The three flashDah() calls signal the letter **O**.
- interLetterGap() creates the required pause.
- The final three flashDit() calls signal the final **S**.

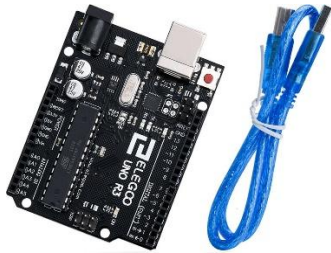
Repetition: The **delay(5000);** creates a long, intentional pause (5 seconds) before the loop() function repeats and the entire S.O.S. signal is sent again. This long pause is crucial for distinguishing one distress call from the next.

This structure allows anyone reading the code to immediately see and understand the logic of the S.O.S. pattern without having to analyze complex for loops or delay calculations.

Mini Lab: Singular LED S.O.S. Morse Code (Hardware Overview)

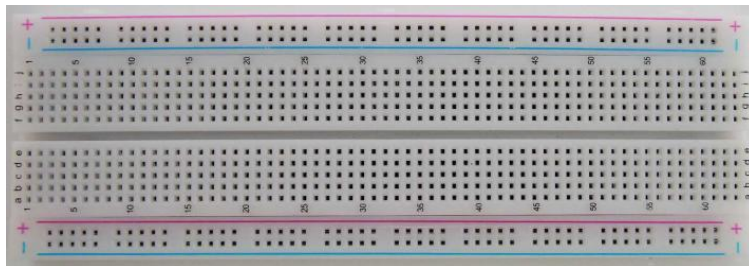
UNO R3 Controller Board: The **Arduino Board** is like the **brain** of your project. It's a small computer that you can program to control the other parts. You write instructions (code) on a regular computer and upload them to the Arduino, and it follows those instructions to make things happen!

Key Function: It runs the program and sends signals to control things.



Breadboard: The **Breadboard** is a plastic board with lots of little holes. It lets you **easily connect** wires and components together without having to permanently solder (glue with metal) them. You just push the wires and component leads into the holes to build the circuit.

Key Function: A temporary, reusable way to connect components and wires.



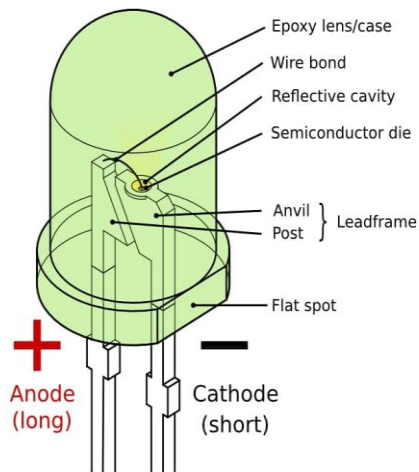
Jumper Wires: **Jumper Wires** are just simple wires used to **connect** all your components and the Arduino board to the breadboard. They complete the circuit so electricity can flow from the Arduino to the components and back again.

Key Function: To easily connect parts on the breadboard and to the Arduino's pins



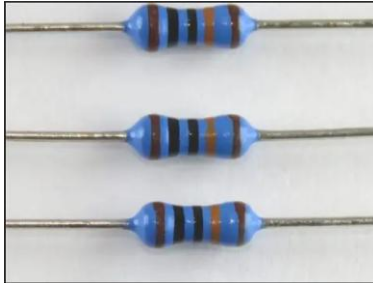
5mm LED: An **LED** is a tiny light bulb that lights up when you send electricity through it. It only allows electricity to flow in **one direction**, so you must connect the longer leg (the Anode or positive side) to the power source and the shorter leg (the Cathode or negative side) to the ground.

Key Function: Converts electrical energy into light.



100-ohm Resistor: A **Resistor** is a small component that **slows down** the flow of electricity (current). When you use an LED, you *must* use a resistor to protect the LED from getting too much electricity and burning out.

Key Function: Limits the amount of electrical current flowing in a circuit.



The value of resistance is known as an Ohm and its symbol is a Greek Omega symbol Ω . In this case, Digital Pin 10 is outputting 5v DC at (according to the Atmega datasheet) 40mA (milliamps), and your LEDs require (according to their datasheet) a voltage of 2v and a current of 35mA. Therefore, you need a resistor that will reduce the 5v to 2v and the current from 40mA to 35mA if you want to display the LED at its maximum brightness. If you want the LED to be dimmer, you could use a higher value of resistance.

Note NEVER use a value of resistor that is LOWER than needed. You will put too much current through the LED and damage it permanently. You could also damage other parts of your circuit.

The formula to work out what resistor you need is

$$R = (V_S - V_L) / I$$

where V_S is the supply voltage, V_L is the LED voltage, and I is the LED current. Your example LED has a voltage of 2v and a current of 35mA connected to a digital pin from an Arduino, which gives out 5 volts, so the resistor value needed would be

$$R = (5 - 2) / 0.035$$

which gives a value of 85.71.

Resistors come in standard values and the closest common value would be 100 Ω . Always choose the next standard value resistor that is HIGHER than the value needed. If you choose a lower value, too much current will flow through the resistor and will damage it.

How Do we Find a Certain Resistor?

A resistor is too small to contain easily readable labeling so resistors instead use a color code. Around the resistor you will typically find 4 colored bands; by using the color code in Table 2-1 you can find out the value of a resistor. Likewise, you can find the color code for a particular resistance.

Color	1st Band (Digit)	2nd Band (Digit)	3rd Band (Multiplier)	4th Band (Tolerance)
BLACK	0	0	10^0 (or x 1)	-
BROWN	1	1	10^1 (or x 10)	±1%
RED	2	2	10^2 (or x 100)	±2%
ORANGE	3	3	10^3 (or x 1,000)	-
YELLOW	4	4	10^4 (or x 10,000)	-
GREEN	5	5	10^5 (or x 100,000)	±0.5%
BLUE	6	6	10^6 (or x 1,000,000)	±0.25%
VIOLET	7	7	10^7	±0.1%
GREY	8	8	10^8	±0.05%
WHITE	9	9	10^9	-
GOLD	-	-	10^{-1}	±5%
SILVER	-	-	10^{-2}	±10%
NONE	-	-	-	±20%

How to Find a 100Ω Resistor?

To get a 100Ω resistor, you need the digits 1 and 0, followed by a multiplier that gives you the total value.

Band	Color	Value	Explanation
1st Band	BROWN	1	The first digit is 1.
2nd Band	BLACK	0	The second digit is 0. And The base value is now 10.
3rd Band	BROWN	10^1	The multiplier tells you how many zeros to add to the base value. $10 \times 10 = 100$.
4th Band	GOLD	±5%	This is the Tolerance , it means the resistor's actual value is very close to 100Ω.